orbital-strike-cannon

Cryptography (crypto)

Challenge

The service prints a JSON telemetry blob and asks for a 32-hex-char **firing code**. On success it returns the flag.

From `cannon.py`, the firing code and ciphertext both come from a SHA-256 key derived from four secrets:

```
secret_material = moon0 || x0 || rng_u || rng_v
key             = SHA256("OSC-KEY|" || secret_material)
firing_code     = SHA256(key || "|fire")[:32]
ciphertext      = SHAKE256(key || "|stream") ⊕ flag
```

Public values include:

- modulus `P = 2^127 − 1`
- octonion parameters `alpha`, `beta`
- scalar `outer_a`
- LCG beacons `rng_beacons`
- 7 satellites (only 5 real), each with linear samples of moon/orbit features
- the ciphertext

Algebra

Everything lives in the octonion algebra over `𝔽_P`, built via Cayley–Dickson from quaternions. Multiplication is **non-associative**; the challenge explicitly uses

```
M_{i+1} = (R_i · M_i) · α  +  β
```

not `R_i · (M_i · α)`.

Orbits update as

```
X_{i+1} = outer_a · X_i + ∑(M_i) + rng_i
```

Each “RNG beacon” octonion `R_i` is a deterministic packing of four consecutive LCG outputs (`rng_octonion`).

The server also builds affine expressions of every later state in terms of the initial secret vector

```
v = (M_0[0], …, M_0[7], X_0, 1)  ∈ 𝔽_P^{10}
```

so each feature vector

```
f_i = (M_i || X_i || X_{i+1} || X_{i+2})  ∈ 𝔽_P^{11}
```

is an affine image of `v`. That is the linear foothold.

Recovering the LCG

`BrokenRNG` is a plain affine congruential generator:

```
s_{n+1} = u · s_n + v  (mod P)
```

All `s_n` are published in `rng_beacons`, so `(u, v)` fall out immediately from three consecutive samples:

```
u = (s₂ − s₁) · (s₁ − s₀)⁻¹
v = s₁ − u · s₀
```

That removes two of the four secrets.

Satellite observations

Each satellite publishes:

- sample indices `arange = [start, stop, step]`
- projection rows `basis` (3×11)
- bias (3)
- coords (5 samples × 3)
- `mask_offset`

For a **real** satellite at index `idx`:

```
mask = rng_beacons[idx + mask_offset]
coord[j] = mask · ⟨basis[j], f_idx⟩ + bias[j]
```

so

```
⟨basis[j], f_idx⟩ = (coord[j] − bias[j]) · mask⁻¹
```

Fake satellites emit random coords and are inconsistent with the state model.

With 5 real satellites × 5 samples × 3 equations, there are plenty of linear constraints on the 9 unknowns `(M_0, X_0)`.

Solving

1. Rebuild the affine state maps with the public `alpha`, `beta`, `outer_a`, beacons (same logic as `build_state_expressions` / `feature_rows` in `cannon.py`).
2. For each satellite, turn its coords into equations on `(M_0, X_0)`.
3. Enumerate all `C(7,5) = 21` choices of which satellites are live.
4. Gaussian-eliminate over `𝔽_P`. Wrong subsets are inconsistent; the correct one yields a unique solution.
5. Recompute `key`, `firing_code`, and decrypt the ciphertext (or submit the firing code to the service).

Remote

Connect with TLS (`ncat --ssl` / `ssl.wrap_socket`), parse the JSON transcript, solve, submit the firing code.

```
LOCK CONFIRMED. CANNON FIRED.
OmniCTF{Wemmbu_h4s_destroy3d_th3_law_hell_nawhh_where-
is_you_eggcha~}
```

Flag

```
OmniCTF{Wemmbu_h4s_destroy3d_th3_law_hell_nawhh_where-
is_you_eggcha~}
```

Takeaways

- Published “broken” RNG beacons make the LCG parameters free.
- Non-associative octonion drama is mostly flavor: the transition is still an $\mathbb{F}_P$-linear map on coordinates once R_i and α are known.
- Dead satellites are a classic consistency filter — enumerate small combinatorial subsets and keep the linear system that closes.